Hikyu Bank Version 6

LCNC Implementation with AI-Powered Workflows — Design Documentation and Reflection

Student: [Your Name] Course: iBPM, Lab Exercise 6 (Module 8) Date: 3 November 2026

Platforms: Saltcorn 1.6.2 (LCNC) • Flowable 7.1.0 (iBPM) • local Naive Bayes text classifier (AI)

Scope. Version 6 is a working local demonstrator. All data is synthetic. Human-task testing used separate accounts operated by one person, not several real people. Complete Version 5 feature parity, public hosting and production readiness are not claimed; the gaps are listed in Step 2.

Step 1 — LCNC platform selection and setup

The lab requires a free-tier LCNC platform that supports user interfaces, a database and application logic. I first planned to use Bubble, but I could not read its pricing or API pages from my environment, so I could not confirm its free-plan limits. I then evaluated Budibase. Its official Docker deployment ran out of disk space while creating the application container, so I removed it rather than report it as working. I finally chose Saltcorn 1.6.2.

Criterion	Saltcorn 1.6.2 (selected)
Cost / licence	Open source (MIT licence); self-hosted at no software cost. Hosting and backups would still cost money for a public site.
User interface	Native page editor, reusable views, role-based home pages (Figure 3).
Database	PostgreSQL-backed tables with fields, ownership and relations (Figure 4).
Application logic	Actions and plugins; security-critical logic is added as source-controlled extensions.
Verification	Installed from the official npm package with a locked dependency manifest; editing, data and workflow tested.

The result is a self-hosted local deployment, not a public website. No paid plan or trial was used.

Step 2 — Replicating the Version 5 application

Version 5 is a Spring Boot 3.5 backend with a JavaScript frontend and a PostgreSQL database (Flyway migrations V1–V8). Before rebuilding, I compiled and tested a clean copy of V5 against a fresh database: 93 backend tests, five frontend test files and 14 startup tests passed, and a browser login/code/PIN flow loaded six routes. I used that run as the reference for the rebuild.

Mapping of the three core components

Component	Version 5	Saltcorn Version 6
UI and layouts	login / register / verify pages; overview, accounts, transactions, transfers, banking tabs, settings, alerts and inbox	Eight native pages with reusable views: overview account cards, Pay & Transfer, Accounts & Cards, Financial Insights, Transactions, Support Requests
Database	users, accounts, transactions, transfers, peer payments, payment verifications, saved payees, card controls, budgets, financial operations, alerts, notifications	Sixteen native tables (including the platform user table) with owner links; one unified event table replaces the normalized notification-recipient and financial-event tables; V5 UUIDs kept separately as source_v5_id

Component	Version 5	Saltcorn Version 6
Logic and workflows	Java services: own-account transfers, peer payments, purchases, repayment, bill pay, card lock, budgets, CSV export, simulated code + PIN authorization	Coded Saltcorn extensions: serialized PostgreSQL transactions, integer-cent amounts, idempotency hashes, authorization codes bound to session and request, salted scrypt PINs, linked ledger legs, CSV formula protection

Implemented and tested: account cards from persisted data, own-account transfers, peer payments, debit/credit purchases, credit repayment, saved billers and bill payment, card lock/unlock, monthly budgets, and CSV statements. Direct user writes to financial and workflow fields are blocked.

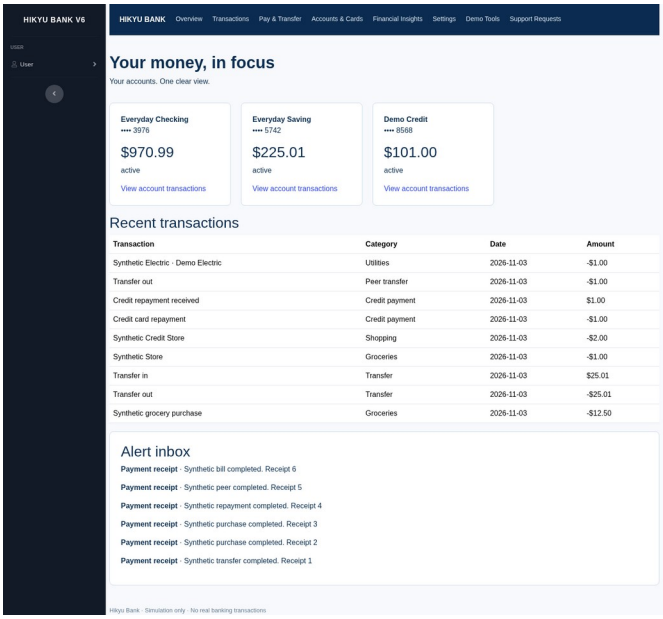


Figure 1. Account cards in the Saltcorn application, read from persisted synthetic data.

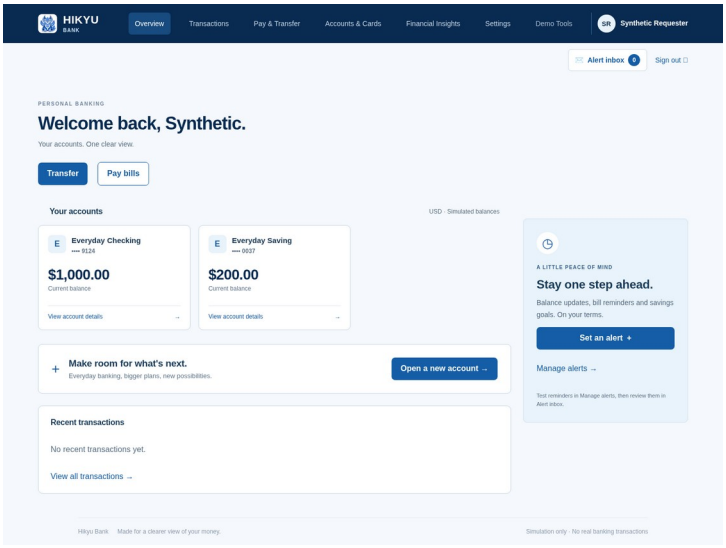


Figure 2. The Version 5 overview used as the visual and functional reference.

Parity gaps (not completed)

- Native email/password sign-in replaces the V5 username/password plus simulated login MFA. Payment code and PIN checks are implemented separately.
- Full registration, profile, security and session management from V5 were not ported.
- The original financial assistant and scheduled alert preferences were not ported.
- Transaction filtering, pagination and detail views are only partly ported; mobile layout parity is not verified.

Development experience

Saltcorn made the data structure, ownership rules and page layouts visible quickly, and its configuration export helps review changes. However, exact money handling, authorization-code binding and transaction atomicity could not be built from visual steps alone; they needed coded extensions and real-database tests. After changes made through the command line, the application had to be reloaded to refresh its metadata. Compared with V5, LCNC was faster for CRUD-style pages and slower for guarantees that depend on transactions and locking. I did not measure a time saving.

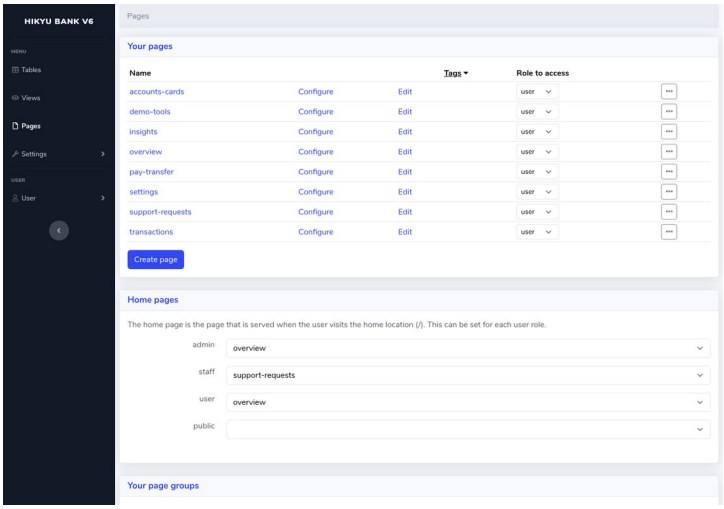


Figure 3. Saltcorn native page editor showing the eight Version 6 pages.

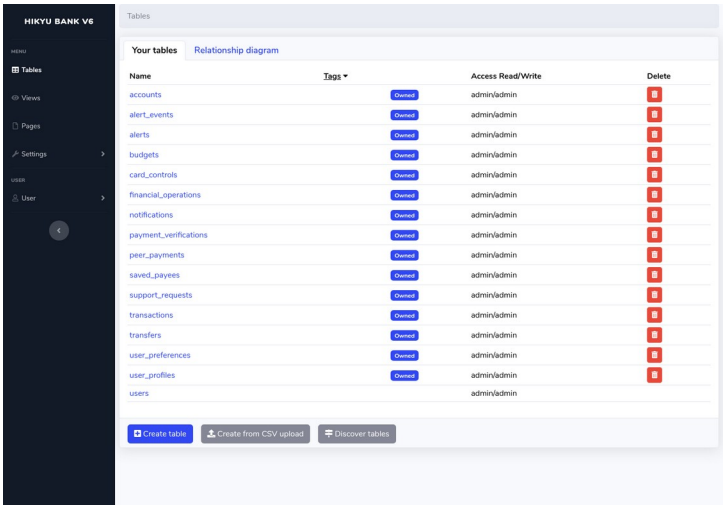


Figure 4. Saltcorn native data editor showing domain tables with ownership fields.

Step 3 — iBPM platform research and selection

I compared two candidates using official repository documentation: Flowable Open Source and n8n Community. Flowable was run and tested; n8n was researched only.

Criterion	Flowable 7.1.0 (selected)	n8n Community
Licence / cost	Apache 2.0; no engine fee	Free Community edition; Sustainable Use licence
Task assignment	BPMN user tasks with assignees and candidate groups	Human-review pause and approve/deny; a role-based task inbox would need extra design
Modelling	Executable BPMN, gateways, service tasks	Visual automation nodes; not BPMN
State and audit	Persistent execution, task and history data	Execution persistence; Community lacks projects, sharing and SSO
Exceptions / timers	Boundary error events and interrupting timers (tested)	Error handling and Wait node; timeout policies must be built
AI	No built-in generative AI; added a real classifier as a service task	Built-in AI nodes; external providers may cost money
Integration	Authenticated REST API	Webhook / HTTP request

Rationale. I chose Flowable because the lab asks for task assignment, status tracking, automation and multiple participants, and BPMN models these directly with durable history and timers. The open-source engine does not include hosted generative AI, so I added AI myself as a BPMN service task that calls a trained classifier. I selected Flowable only after the chain Saltcorn → AI → agent/reviewer handoff → write-back to Saltcorn passed in a live test. Sources: Flowable README and LICENSE (github.com/flowable/flowable-engine); n8n documentation (github.com/n8n-io/n8n-docs: community edition features, human-in-the-loop for tools, license FAQ); Saltcorn README and LICENSE (github.com/saltcorn/saltcorn). Snapshots are in the research/ folder.

Step 4 — Business process workflow

Business case: Transaction Support Review. A customer selects one of their own transactions and describes a problem (for example, a locked card). The workflow routes the request through AI categorization and two staff roles. It never changes a settled transaction or makes a credit decision.

Participant	Task	Hands off to
Requester	Select an owned transaction, describe the issue, supply missing information if returned	Service agent
Service agent	Review the AI category and context; continue, return or reject	Reviewer, requester or end
Reviewer	Independently complete or reject the handling (must differ from the agent)	Final result
Supervisor	Handle a review that passed its deadline	Final result

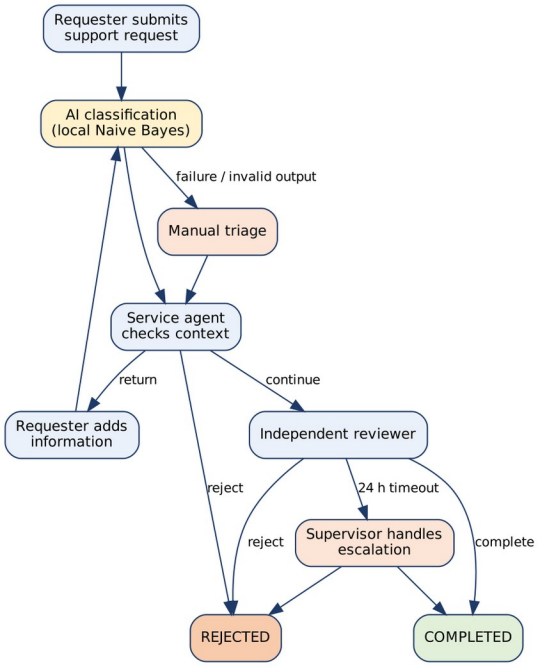


Figure 5. Transaction Support Review process (BPMN definition: implementation/workflow/src/main/resources/transaction-support.bpmn20.xml).

Required workflow elements

- **Task assignment:** Flowable user tasks. The service checks the authenticated role, record ownership, active task and expected version; candidate groups alone are not trusted.
- **Workflow automation:** Gateways route continue, return and reject; failures route to manual triage; a timer escalates overdue reviews.
- **Status tracking:** Displayed states are SUBMITTED, MANUAL_TRIAGE, SERVICE_REVIEW, RETURNED, REVIEWER_REVIEW, ESCALATED, COMPLETED and REJECTED. The state is written to the same Saltcorn record and survives a refresh. Final states cannot be overwritten by late updates.
- **AI-powered feature:** a multinomial Naive Bayes classifier assigns card_access, payment_tracking or general_support (unclassified for unknown words). It is trained on 24 synthetic sentences; all six holdout examples were classified as expected, which is not a production accuracy estimate. The BPMN service task starts the Python classifier at runtime, sends only the description text, validates the JSON result and stores input and output as evidence. It cannot approve anything or move funds.

Workflow testing

I tested that tasks move correctly between participants and that automation behaves as intended, using separate role accounts and browser sessions. The normal reviewer deadline is 24 hours; to test escalation I used a two-second deadline, which caused a real timer expiry and an escalation to the supervisor. I did not observe a 24-hour wait. AI failures (timeout, malformed output, process failure) were injected deliberately and all reached MANUAL_TRIAGE.



Figure 6. Submitted request: AI category, confidence, model version, SERVICE_REVIEW stage and process instance are saved on the record.



Figure 7. The same record after handoff from service agent to independent reviewer. A decision comment is required; the AI does not approve.

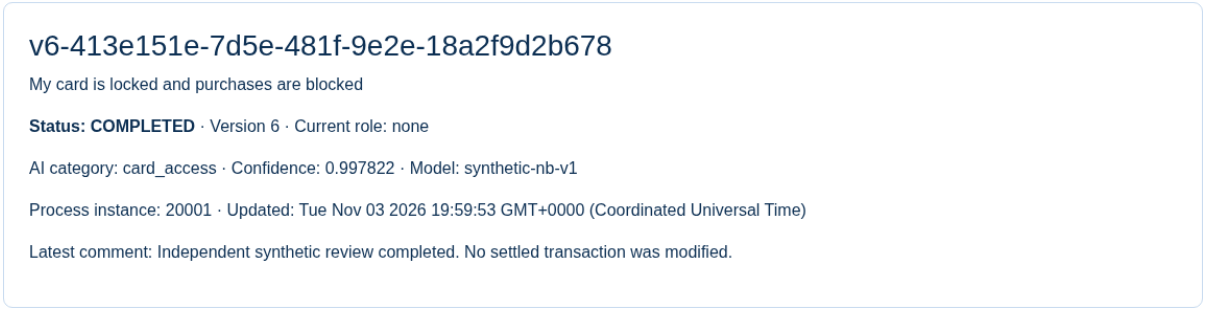


Figure 8. The same record after reviewer completion and a page refresh: COMPLETED, a later version, the saved comment and the same process instance.

Step 5 — Version 6 integration and design

Version 6 combines the Saltcorn application from Step 2 with the Flowable workflow from Step 4. Saltcorn owns the business record and the financial ledger. Flowable owns execution, tasks, timers and history; its status is a projection, never a second banking database.

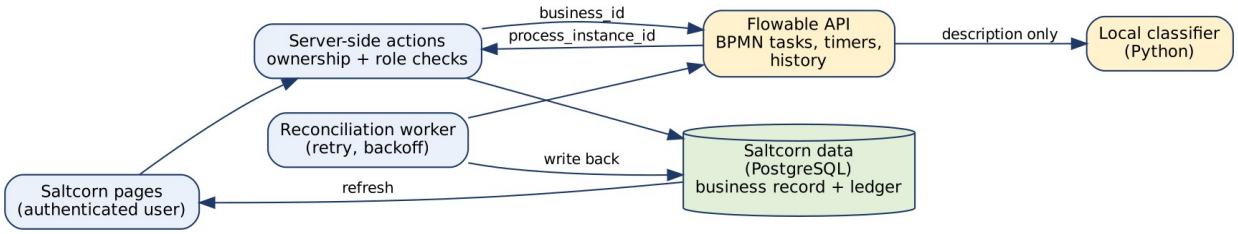


Figure 9. Version 6 architecture.

Integration design

- Saltcorn creates the business record first and sends its stable business_id to the Flowable API; Flowable returns a process_instance_id, and AI category, stage, sequence, time and comments are written back to the same record.
- The API contract is in implementation/openapi.yaml (submit process, read status, task decisions).
- Duplicate requests with the same content return the original record; changed content under the same key returns a conflict; stale task versions are rejected.
- A background worker retries failed submissions and write-backs with exponential backoff (capped at 60 seconds). I tested this by stopping the workflow API, adding a temporary database constraint, and restoring both; the record converged without manual repair.
- Browsers receive normal session cookies and CSRF protection; workflow service tokens and passwords stay on the server. Submitted text is treated as data, not instructions.
- Integration is local loopback HTTP with polling. A public deployment would need HTTPS, managed secrets and hardened authentication.

Problems found and how I solved them

Problem	Resolution
Budibase deployment exhausted disk space	Removed it and switched to Saltcorn after a successful install and workflow test
Visual updates could not guarantee atomic money movement	Wrote coded extensions with PostgreSQL transactions, locking and idempotency, and tested them against the real database
A SQL keyword collision and a test-harness CSRF issue	Fixed both and re-ran the affected tests
Flowable does not enforce all candidate-group permissions by itself	Added a wrapper that checks role, ownership, active task and version
Failed callbacks left stale status	Added durable pending records and a reconciliation worker

Test results

Area	Result
V5 baseline (fresh database)	93 backend tests, 5 frontend test files, 14 startup tests passed
Live Saltcorn → Flowable human workflow	8 end-to-end checks and 10 resilience/security checks passed
Banking reconstruction	14 banking checks and 5 CSV checks passed
AI failure and timer tests	3 fault-injection checks and 1 accelerated timer escalation passed
Not done	Real multiple-person testing, public hosting, restoring onto a separate installation, full V5 parity

Scripts for every check are in implementation/ and the recorded outputs are in evidence/.

Governance and ethics

The system uses synthetic data only, minimal model input (no names, PINs, balances or demographics), mandatory human review, a bounded label set, record ownership, task history, idempotency and monotonic state versions. Remaining concerns: weak generalization from a tiny

training set, uncalibrated confidence, incomplete V5 authentication parity, upkeep of the code extensions, loopback-only HTTP and untested production load. The demonstrator is not suitable for real banking or automated credit decisions.

Step 6 — Reflection

Adding the AI classifier gave reviewers a starting category for every support request, so the service agent no longer began from a blank description. Refreshing the page showed the same record moving through each stage, which made status easy to follow. I did not measure time savings, and the model was trained on only 24 synthetic sentences, so I make no accuracy or efficiency claims.

Rebuilding in Saltcorn made data and pages quick to inspect, but exact money handling, authorization-code binding and retry recovery still needed custom code and tests. Automation also hid responsibility inside less visible components, so I added human review, minimal model input (description text only), role checks, task history and escalation timers. I would keep these controls even if they slow the process.

AI acknowledgement: I used Claude (Anthropic) to help generate code and tests and to draft this documentation. I ran and reviewed the results and take responsibility for the submission.

(156 words)

Appendix — Running Version 6

See README.txt in the submission folder for the same steps. Prerequisites: Node.js, Python 3, Docker and a JDK 17 or later.

1. Check the environment: python3 implementation/prepare_v6.py --check
2. Prepare it: python3 implementation/prepare_v6.py (creates new private secrets and databases)
3. Start the database: docker compose -p hikyu-v6 -f implementation/compose.v6.yaml up -d --wait
4. Start services: python3 implementation/start-services.py (workflow API on port 8096, Saltcorn on 8097)
5. Open http://localhost:8097/auth/login, sign in with a generated demo account, and open Support Requests
6. Submit a synthetic request as Requester, hand it on as Service agent, then complete it as Reviewer; refresh as Requester to see the final state
7. Re-run tests: node implementation/saltcorn/test_e2e.cjs (also test_resilience, test_banking, test_csv) and python3 implementation/test_ai_faults.py and test_sla.py

The LCNC configuration export is deliverables/Saltcorn_V6_Application_Pack.json. It contains definitions only, no passwords or business rows. Restoring it on a separate installation was not tested. Passwords, PINs and service tokens are generated locally and are not included.